Computer Engineering Senior Project Proposal

B.Eng. Computer Engineering

Academic Year 2568

Project Title

Development of an AI-Based Multi-UAV with Cooperative Search Algorithms for Human Detection and Location Reporting in Post-Flood Environments

Members

1. Chayuthpong Karnkonsue student ID 66070501011
2. Nitis Jadipang student ID 66070501020
3. Tanapat Phomak student ID 66070501023
4. Nattakarn Phoonet student ID 66070501074

Advisor: Dr. Prapong Prechaprapranwong

Signature: _____ (Advisor)

(Dr. Prapong Prechaprapranwong)

Table of Content

Section 1 1

1.1 Project introduce & problem statement 1

1.2 Objectives 2

1.3 Scope 2

1.3.1 Solutions 2

1.3.2 Feature 3

1.3.3 Target Users and Operating Environment 3

1.4 Project Plan 4

1.4.1 First Semester 4

1.4.2 Second Semester 5

1.5 Gantt chart 7

1.5.1 First Semester 7

1.5.2 Second Semester 8

1.6 Deliverable 9

1.6.1 First Semester 9

1.6.2 Second Semester 9

Section 2 10

2.1 Background 10

2.1.1 Operational Workflows for UAVs in Flood SAR Missions 10

2.1.2 Theoretical Foundations and Core Algorithms 11

2.1.3 System Infrastructure and Frameworks 15

2.1.4 Existing Solutions 19

2.2 Preliminary design based on backgrounds 21

Section 1

1.1 Project introduce & problem statement

Flooding is among the most destructive natural hazards in Thailand, causing severe human casualties and infrastructure failure [1]. During the critical 72-hour Golden Window, rapid survivor localization is paramount before threats of drowning and severe physical exposure escalate [2], [3]. However, hazardous post-flood environments characterized by high-velocity currents, submerged structural wreckage, damaged utility poles, and communication blackouts severely obstruct boat and ground teams, forcing responders into hazardous, slow manual blind searches [4].

While Unmanned Aerial Vehicles (UAVs) provide elevated situational awareness without placing personnel directly into hazardous waters [5], their practical deployment in emergency flood response is hindered by two core engineering bottlenecks:

1. **Manual Visual Observation Bottleneck:** Conventional operations rely on ground operators manually screening live aerial video feeds on ground station monitors. This process imposes severe cognitive fatigue and carries a high risk of target omission across visually complex, reflective, and turbid water environments [5], [6].
2. **Georeferencing Disconnect (2D Image to 3D Physical Space):** Standard vision models yield only 2D bounding boxes on an image plane. Without continuous geometric transformation using flight telemetry and surface boundary constraints, visual detections cannot be translated into actionable geographic coordinates required by field rescue vessels [6].

To address these limitations, this project develops an end-to-end automated spatial data pipeline and mission supervision platform for multi-UAV flood search operations. The proposed system integrates fleet-size adaptive mCPP, battery-aware dynamic replanning, AI-based survivor detection, high-frequency telemetry streaming, and ray-plane georeferencing into a centralized tactical mapping dashboard. By autonomously filtering visual noise and transforming aerial detections into spatial database records, the system provides real-time geographic mapping for field commanders. This approach shifts post-flood operations from manual blind searches to a data-directed rescue model [3].

1.2 Objectives

1. To design and implement a scalable Multi-UAV Coverage Path Planning (mCPP) system

that adaptively partitions and allocates search areas across varying fleet sizes for flood search missions.

2. To develop an adaptive dynamic replanning mechanism capable of monitoring UAV battery states and automatically re-allocating remaining search.
3. To implement an automated survivor detection and georeferencing engine to compute real-world geographic coordinates.
4. To develop a centralized monitoring dashboard for defining mission boundaries, tracking fleet status and telemetry in real time, and visualizing detected survivor incidents with image evidence and generating post mission summary analytics for tactical handover and debriefing.
5. To evaluate overall system performance across varying fleet sizes, focusing on area coverage efficiency, mission completion time, dynamic replanning resilience under battery constraints, and target georeferencing accuracy under simulated environments.
6. To validate a physical proof-of-concept by deploying the developed system onto an actual UAV platform using a standardized software interface, demonstrating reusable integration across compatible drones equipped with onboard vision, GPS, and flight-control capabilities under controlled test environments.

1.3 Scope

1.3.1 Solutions

This project aims to develop an end-to-end Multi-UAV system for post-flood search and rescue (SAR). It integrates adaptive Multi-UAV Coverage Path Planning (mCPP), real-time AI human detection, and dynamic fleet battery management to autonomously locate stranded individuals and instantly calculate their real-world geographic coordinates.

1.3.2 Feature

- **Search and Coverage Path Planning (mCPP):** An automated engine that dynamically partitions a user-defined search area into N balanced, non-overlapping trajectories based on the number of active drones.

- **Adaptive Dynamic Replanning:** A runtime supervisor that monitors individual drone battery states. If a unit triggers Return-to-Launch (RTL) due to power depletion, the module automatically redistributes unvisited waypoints among the remaining fleet to ensure 100% coverage.
- **AI Human Detection & Georeferencing Engine:** Uses YOLO to detect and classify humans from live video streams. A ray-plane intersection algorithm projects 2D image bounding box centroids into 3D geographic coordinates using synchronized camera and drone telemetry.
- **Centralized Monitoring Dashboard:** A Grafana-based interface visualizing live telemetry (battery, altitude, position). It also generates post-mission analytics, including area coverage percentages, total duration, and a verified survivor ledger with coordinates and snapshots.
- **Telemetry & Video Streaming Pipeline:** Transmits live video to a central server alongside MQTT-based telemetry and control messages.
- **Simulation & Hardware Implementation:** Algorithms are rigorously tested in Gazebo, using PX4 for flight control and ROS 2 for communication, before deployment onto physical UAV platforms via reusable, standardized software interfaces.

1.3.3 Target Users and Operating Environment

The primary target users are tactical SAR incident commanders and UAV fleet operators at the command post. To prevent visual fatigue from continuous aerial surveillance, the system acts as an assistant that autonomously flags candidate targets and computes coordinates, allowing operators to focus on rapid cross-verification rather than exhaustive visual scanning.

The secondary target group comprises field rescue boat units; while they do not interact with the software pipeline directly, they utilize the verified geographic coordinates and confirmation snapshots to execute direct, point-to-point survivor extractions instead of conducting perilous, unguided water searches.

1.4 Project Plan

1.4.1 First Semester

1.4.1.1 Research

- Research search and rescue (SAR) operational workflows in post-flood disaster scenarios.
- Identify the objective and scope of the project.
- Study fundamental concepts and select development tools.

1.4.1.2 Proposal

- Proposal report.
- Project Progress Presentation Preparation.

1.4.1.3 Simulation Environment Setup

- Configure and set up simulation environment.
- Establish inter-process communication bridges between simulated aerial vehicles and ground computing services.

1.4.1.4 Data Pipeline and Streaming Infrastructure

- Design the distributed streaming architecture for concurrent aerial-to-ground telemetry and video transmission.
- Develop data pipelines for distributed streaming architecture.
- Design the schema and deploy a spatial-temporal database.

1.4.1.5 Scalable Multi-UAV Coverage Path Planning (mCPP) Algorithm

- Design and develop parameterized coverage trajectory for arbitrary numbers of active vehicles.
- Validate baseline path generation and workload balancing.

1.4.1.6 AI-Based Human Detection Model

- Collect and preprocess annotated aerial imagery.

- Develop AI human detection model.
- Evaluate detection performance metrics.

1.4.1.7 Baseline Fleet Monitoring Prototype

- Design and implement a prototype monitoring interface to ingest and display real-time drone telemetry.
- Visualize simulated multi-vehicle flight paths, coordinates, and battery states to validate coverage path planning execution.

1.4.1.8 Project Progress

- Project report.
- Progress Presentation Preparation.

1.4.2 Second Semester

1.4.2.1 Physical Drone Interface Preparation

- Prepare the physical UAV testbed and verify onboard sensor payloads, software communication interfaces and protocol parsers.
- Conduct baseline bench tests to verify live telemetry and video feed transmission from physical hardware to ground ingestion services.

1.4.2.2 Spatial Computing and Georeferencing Engine

- Implement the ray-plane intersection algorithm to project 2D image bounding box centroids.
- Integrate the object detection inference service with the georeferencing engine to output real-world geographic coordinates.

1.4.2.3 Adaptive Dynamic Replanning and Fleet Supervisor

- Develop an adaptive dynamic replanning engine to handle fleet topological changes.
- Validate dynamic replanning engine.

1.4.2.4 Centralized Tactical and Mission Debrief Dashboard

- Integrate spatial survivor detection pins into the live dashboard.
- Implement the post-mission summary module.

1.4.2.5 System Integration and Hardware Validation

- Execute end-to-end software integration.
- Conduct controlled physical proof-of-concept (PoC) flight tests with the physical UAV platform.

1.4.2.6 Performance Evaluation and Benchmarking

- Benchmark multi-UAV coverage efficiency, workload balance, and total mission completion times.
- Quantify dynamic replanning performance.
- Evaluate target georeferencing accuracy.
- Benchmark AI human detection model.

1.4.2.7 Project Report

- Project report preparation.
- Project Presentation.

1.5 Gantt chart

1.5.1 First Semester

1. Research		
1.1 Research SAR workflows.	Aug – Sep	All
1.2 Identify the objective and scope	Sep	All
1.3 Study concepts and tools	Sep	All
2. Proposal		
2.1 Proposal Report	Sep	All
2.2 Project Progress Presentation Preparation	Sep	All
3. Simulation Environment Setup		
3.1 Configure and set up simulation environment.	Oct	Tanapat
3.2 Establish communication	Oct	Tanapat
4. Data Pipeline and Streaming Infrastructure		
4.1 Design the distributed streaming architecture	Oct	Chayuthpong, Tanapat
4.2 Develop data pipeline	Oct	Chayuthpong
4.4 Design and deploy database	Oct	Chayuthpong
5. Scalable Multi-UAV Coverage Path Planning (mCPP) Algorithm		
5.1 Design and Develop algorithm	Oct – Nov	Nitis
5.2 Test algorithm	Nov	Nitis
6. AI-Based Human Detection Model		
6.1 Datasets Preparation	Nov	Chayuthpong, Nitis
6.2 Develop models	Nov	Nitis
6.3 Evaluate detection metrics	Nov	Nitis
7. Baseline Fleet Monitoring Prototype		
7.1 Visualize real-time monitoring drone fleet	Nov – Dec	Chayuthpong
7.2 Visualize simulated multi-vehicle flight trajectories and coverage boundaries	Nov – Dec	Chayuthpong
7.3 Validate end-to-end data pipeline connectivity	Dec	Chayuthpong
8. Project Progress		
8.1 Project report	Dec	All
8.2 Progress Presentation	Dec	All

Table 2: First Semester Gantt Chart (subtask / approximate timeframe / owner)

1.5.2 Second Semester

1. Physical Drone Interface Preparation		
1.1 Prepare the physical UAV	Jan	Nattakarn
1.2 Verify transmission data from physical hardware	Jan – Feb	Nattakarn
2. Spatial Computing and Georeferencing Engine		
2.1 Implement the ray-plane intersection algorithm	Feb	Chayuthpong
2.2 Integrate with the object detection	Feb	Chayuthpong
3. Adaptive Dynamic Replanning and Fleet Supervisor		
3.1 Develop an adaptive dynamic replanning engine	Feb – Mar	Nitis
3.2 Validate dynamic replanning engine.	Mar	Nitis
4. Centralized Tactical and Mission Debrief Dashboard		
4.1 Integrate spatial survivor pins into the live dashboard	Mar	Chayuthpong, Tanapat
4.2 Implement the post-mission summary module	Mar	Chayuthpong
5. System Integration and Hardware Validation		
5.1 Execute end-to-end software integration	Apr	Tanapat, Nattakarn
5.2 Physical PoC	Apr	Tanapat, Nattakarn
6. Performance Evaluation and Benchmarking		
6.1 Benchmark multi-UAV coverage efficiency	Apr – May	Nattakarn
6.2 Quantify dynamic replanning performance	Apr – May	Nattakarn
6.3 Evaluate target georeferencing accuracy	Apr – May	Nattakarn
6.4 Benchmark the AI human detection model	Apr – May	Nattakarn
7. Project Report		
7.1 Project report	May	All
7.2 Project Presentation	May	All

Table 4: Second Semester Gantt Chart (subtask / approximate timeframe / owner)

Note: the original document renders these as week-by-week shaded grids (Aug–Dec and Jan–May, four weeks per month). The tables above preserve every subtask, its approximate timeframe, and its owner; replace the "timeframe" column with the exact weekly grid if you need the identical visual layout.

1.6 Deliverable

1.6.1 First Semester

- Proposal report.
- Proposal presentation.
- Multi-UAV Simulation Environment.
- Distributed Streaming & Spatial Database Infrastructure.
- Scalable mCPP Algorithm.
- AI Human Detection Models.
- Baseline Fleet Monitoring Prototype.

1.6.2 Second Semester

- Ray-Plane Georeferencing Engine.
- Adaptive Dynamic Replanning Engine.
- Monitoring and Post-mission Dashboard.
- End-to-End Integrated System & Physical PoC Validation.
- Experimental Benchmarking Report.
- Final Project Report.

Section 2

2.1 Background

2.1.1 Operational Workflows for UAVs in Flood SAR Missions

Search and Rescue (SAR) operations following severe flood events are highly time-sensitive, governed by the critical 72-hour survival window where the probability of rescuing living victims is highest [3], [4]. Post-flood environments pose severe logistical hazards for traditional ground and boat rescue teams, including high-velocity water currents, submerged infrastructure, debris, and downed electrical lines [2]. While deploying a single Unmanned Aerial Vehicle (UAV) provides a safer, elevated vantage point [5], traditional manual drone operations require operators to exhaustively monitor live video feeds. This manual approach is unscalable for widespread disasters, as operators rapidly suffer from cognitive fatigue, increasing the risk of missing stranded survivors amidst complex, debris-filled floodwaters [6].

2.1.1.1 Multi-UAV Coordination and Aerial Fleet Constraints

To overcome the limitations of a single UAV, namely limited battery capacity (typically 20–30 minutes of flight time) and restricted area coverage [5], SAR operations increasingly utilize Multi-UAV fleets. Deploying multiple drones allows for the parallelization of search tasks, significantly reducing the time required to sweep a large region [7]. However, multi-UAV coordination introduces strict operational constraints. The system must ensure that drones do not overlap their search boundaries (to maximize efficiency) and do not cross paths (to avoid collisions). Furthermore, aerial fleets are dynamic; drones consume battery at different rates depending on wind resistance and payload. A robust multi-UAV system must be capable of adapting on the fly, autonomously reassigning search zones if a unit drops offline or triggers a Return-to-Launch (RTL) protocol [7].

2.1.1.2 Autonomous Swarm Workflows

In time-critical post-flood environments, Search and Rescue (SAR) operations benefit significantly from transitioning from manual visual searches to autonomous, multi-agent swarm workflows [3], [4]. To efficiently scale these operations, modern systems can be modeled on formal scenario description languages for unmanned swarms, such as SL4U [8].

As illustrated in the cooperative swarm framework (Figure 2.1), operations are divided into distinct but interconnected phases: an autonomous aerial Search Team and a physical Rescue

Team [8]. The Search Team is deployed to rapidly scan the designated area; if no target is found, it returns to base [8]. However, upon positive target identification, the system immediately triggers the deployment of the Rescue Team to execute a direct, point-to-point extraction, while the Search Team concludes its flight [8]. This multi-team architecture provides the foundational background for replacing hazardous blind searches with a coordinated, data-driven rescue interception.

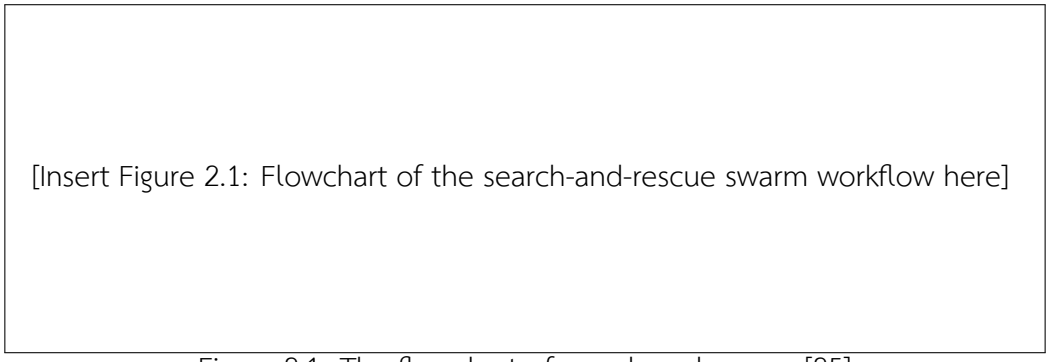


Figure 2.1: The flowchart of search and rescue [25]

2.1.2 Theoretical Foundations and Core Algorithms

2.1.2.1 Multi-UAV Coverage Path Planning (mCPP) algorithm

Coverage Path Planning (CPP) is a computational geometry problem that determines a path passing through all points of a given area or volume while avoiding obstacles [7]. In a Multi-UAV context (mCPP), the algorithm must first partition a user-defined global search area into N distinct sub-regions, where N represents the number of active drones. This is typically achieved using area decomposition techniques, such as grid-based partitioning or Boustrophedon cellular decomposition, which balance the workload evenly across the fleet [7]. Once partitioned, the algorithm generates deterministic, non-overlapping trajectories, most commonly "lawn-mower" or zigzag patterns, tailored to the specific geometry of each sub-region. If the fleet size changes due to battery constraints, the mCPP algorithm must recalculate the remaining unsearched spatial graph and dynamically re-partition the area among the surviving drones to guarantee 100 percent geographic coverage [7].

2.1.2.2 YOLO-Based Object Detection for UAVs

You Only Look Once (YOLO) is a state-of-the-art, single-stage object detection architecture highly optimized for real-time computer vision [9]. Unlike two-stage detectors (like Faster R-CNN) that first propose regions of interest and then classify them, YOLO frames object detection as a single regression problem. It divides the input image into a grid and simultaneously predicts

bounding box coordinates and class probabilities for each grid cell [9]. This unified architecture dramatically reduces inference latency, making YOLO exceptionally suited for processing high-frequency live video streams transmitted from moving UAVs [6]. In modern UAV-based SAR literature, YOLO is frequently utilized to autonomously isolate and classify human targets from high-altitude aerial feeds, flagging potential survivors faster than a human operator could perceive them.

2.1.2.3 Projective Geometry & Ray-Plane Georeferencing

Projective Geometry and Camera Calibration

Projective geometry describes the mathematical relationship between a 3D point in the real world and its 2D projection onto an image plane. The camera projection model can be expressed as

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} \tag{1}$$

where u and v are image pixel coordinates, X_c, Y_c, Z_c are coordinates in the camera frame, K is the camera intrinsic matrix, and s is a scale factor. The intrinsic matrix is

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \tag{2}$$

where f_x and f_y represent the focal lengths and c_x, c_y represent the principal point. Camera calibration provides these intrinsic parameters, which are required to determine the viewing direction of a detected object [7], [9].

Pixel-to-Ray Transformation

A point in an image can be represented by its pixel coordinates (u, v). These coordinates can be transformed into a 3D viewing ray in the camera coordinate system using the inverse of the intrinsic matrix:

$$r_c = K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \tag{3}$$

where r_c represents the direction of the viewing ray in the camera coordinate system. The viewing ray can then be transformed into a world coordinate system using a rotation matrix:

$$r_w = R r_c \tag{4}$$

where R represents the rotation between the camera and world coordinate systems and r_w represents the ray direction in the world coordinate system. The position of a point along the ray is represented as

$$P(t) = C + tr_w \tag{5}$$

where C is the camera position and t represents the distance along the ray [23], [7].

Ray-Plane Intersection

Ray-plane intersection is a geometric method for determining the position of a point by finding the intersection between a viewing ray and a defined planar surface. A plane in three-dimensional space can be represented by

$$ax + by + cz + d = 0 \tag{6}$$

The intersection point is obtained by determining the value of t for which $P(t) = C + tr_w$ satisfies the plane equation. The resulting point represents the estimated 3D position of the detected person on the reference surface.

For surfaces that cannot be adequately represented by a single plane, the same principle can be extended by using a three-dimensional surface or elevation model. Ray-based intersection with three-dimensional terrain has been applied in image-based geolocation research [7], [9].

Georeferencing in Post-Flood Environments

Georeferencing is the process of relating a position in an image or local coordinate system to a geographic coordinate system. The general geometric transformation can be represented as

$$(u, v) \rightarrow r_c \rightarrow r_w \rightarrow P(X, Y, Z) \rightarrow (\text{Latitude}, \text{Longitude}) \tag{7}$$

The accuracy of this process depends on the accuracy of the camera calibration, coordinate transformations, camera position and orientation, and the reference surface used for determining the three-dimensional position. These geometric principles form the basis of image-based geolocation methods [7], [9].

2.1.2.4 Distributed Data Streaming

Fundamentals of Distributed Message Systems

Distributed message systems enable communication between independent applications by exchanging structured data through an intermediary message broker [10]. A foundational architecture is the Publish/Subscribe (Pub/Sub) pattern, where publishers dispatch messages to designated topics without direct knowledge of receiving nodes, and subscribers consume payloads from topics of interest. This architectural pattern decouples senders and receivers across both space and time while enabling non-blocking, asynchronous communication [10].

A critical characteristic of these systems is the enforcement of delivery guarantees, which govern message transmission reliability [10]. Standard models include at-most-once, where messages may be dropped during network degradation but are never duplicated; at-least-once, which guarantees message arrival via retries at the expense of potential duplicates; and exactly-once, which ensures each message is processed without loss or duplication through coordinated state management [10]. These guarantees allow system designers to establish an optimal trade-off among protocol throughput, delivery latency, and implementation complexity [11].

Message Queuing Telemetry Transport (MQTT) Protocol

Message Queuing Telemetry Transport (MQTT) is a lightweight messaging protocol designed for communication in environments with limited bandwidth and constrained devices, making it particularly suitable for Machine-to-Machine (M2M) and Internet of Things (IoT) applications [11]. MQTT implements the publish/subscribe model through a broker, which manages message distribution between clients. Publishers send messages to specific topics, while subscribers receive messages by subscribing to those topics [12].

Furthermore, MQTT provides three configurable Quality of Service (QoS) levels to balance message delivery reliability and communication overhead [11]. QoS 0 (At most once) delivers a message without requiring an acknowledgment. QoS 1 (At least once) uses acknowledgment and retransmission to ensure that the message is delivered, although duplicate messages may occur. QoS 2 (Exactly once) uses a four-step handshake to ensure that each message is delivered and processed only once [11]. These QoS levels allow applications to select an appropriate level of delivery reliability according to their requirements and network conditions [11].

2.1.3 System Infrastructure and Frameworks

2.1.3.1 Distributed Data Streaming Infrastructure

Apache Kafka Architecture and Stream Processing

Apache Kafka is a distributed event streaming platform designed to handle high-throughput, scalable, and fault-tolerant data streams [13]. Kafka organizes events into topics, which are divided into multiple partitions [13]. Partitions allow events to be distributed across multiple brokers, enabling parallel data processing and horizontal scalability [10,13]. Kafka also replicates partitions across brokers to improve fault tolerance and maintain data availability in the event of broker failures [13].

Kafka consists of producers, brokers, and consumers [13]. Producers publish events to Kafka topics, while consumers retrieve and process these events asynchronously [10,13]. Consumers can be organized into consumer groups, allowing multiple consumer instances to share the processing workload of a topic's partitions [13]. This architecture enables Kafka to efficiently handle large volumes of continuous data while allowing events to be retained and consumed at different rates [13].

For stream processing, Kafka provides Kafka Streams, a client library for building applications that process and analyze data continuously as events are received [14]. It supports operations such as filtering, transformation, aggregation, joining, and windowing of event streams [14]. Kafka Streams uses Kafka topics and partitions as the foundation for distributed processing, enabling applications to scale while maintaining processing state and fault tolerance [13], [14].

2.1.3.2 Time-Series Databases

Time-Series Data Management for High-Frequency Telemetry

Time-series data consists of data points recorded sequentially over time, where each measurement is associated with an explicit timestamp [15]. In high-frequency telemetry systems, sensors and edge devices continuously generate large volumes of timestamped records, including geographic coordinates, environmental metrics, and hardware operational states [15]. Managing these data streams requires specialized database architectures, as standard relational storage engines experience severe write degradation under high-ingestion workloads when primary index structures exceed available physical memory [15]. Consequently, systems designed for high-frequency telemetry prioritize high-throughput append-only writes, automated partitioning by temporal ranges, and efficient aggregation mechanisms over specific time windows [15].

TimescaleDB

TimescaleDB is an open-source time-series database engineered as an extension of PostgreSQL, combining native time-series data management capabilities with full relational database features and standard SQL support [15], [16]. TimescaleDB introduces an architectural abstraction known as hypertables, which present themselves to client applications as single standard tables while internally partitioning data across two dimensions: time intervals and optional space partitions [16]. These partitions are materialized as isolated physical tables termed chunks [16]. By dynamically sizing chunks to fit within system memory (RAM), TimescaleDB maintains steady high-throughput ingestion rates without I/O thrashing, while providing native retention policies, analytical window functions, and transparent integration with existing relational schemas [15], [16].

2.1.3.3 Spatial Database

Relational Spatial Databases

Spatial databases are specialized data management systems designed to store, manipulate, and query geometric and geographic entities, such as points, linestrings, and polygons [17]. Unlike conventional alphanumeric relational schemas, spatial data models represent entities situated within multidimensional coordinate reference systems, requiring specialized operations to evaluate topological predicates such as containment, intersection, adjacency, and Euclidean proximity [17]. To support these operations efficiently, spatial database engines implement standardized spatial data types, coordinate transformation functions, and geometric algorithms adhering to Open Geospatial Consortium (OGC) specifications [17].

Geospatial Indexing

Geospatial indexing structures are required to maintain deterministic query execution times over dense geographic datasets. Conventional one-dimensional B-tree indexes cannot preserve multidimensional spatial locality, failing to index multidimensional coordinate spaces effectively [17], [18]. To resolve this, spatial engines implement tree-based bounding structures, notably the Generalized Search Tree (GiST) [18]. GiST partitions coordinate spaces hierarchically using Minimum Bounding Boxes (MBRs), executing queries through a two-step evaluation: an initial bounding-box filter phase that swiftly discards non-candidate geometries, followed by an exact refinement phase that executes rigorous mathematical predicates on candidate geometries [18]. This hierarchical pruning drastically minimizes unnecessary disk I/O and computing overhead during complex boundary intersections [18].

PostGIS

PostGIS is an open-source spatial extension for PostgreSQL that integrates geospatial storage, multidimensional indexing, and spatial analysis natively into a relational database management system [19]. PostGIS introduces the geometry (planar Euclidean) and geography (geodetic spherical) data types, providing an extensive suite of spatial functions adhering to SQL/MM standards for distance calculation, dynamic buffer generation, and topological union operations [17], [19]. Furthermore, PostGIS natively incorporates GiST indexing, delivering scalable spatial query performance across large-scale vector geometries [18], [19]. This architectural integration allows systems to query complex spatial geometries alongside transactional attributes and time-series telemetry within a unified SQL environment [19].

2.1.3.4 Robotics Simulation and Middleware

3D Physics Simulation (Gazebo Simulator)

Gazebo is a physics-based robotics simulator used to develop and evaluate robotic systems in virtual environments. It provides simulation of robot dynamics, sensors, environments, and interactions with simulated objects, allowing robotic algorithms to be tested without requiring physical hardware. This is particularly useful for UAV research because real-world flight experiments can be costly, time-consuming, and subject to safety and hardware constraints. Previous research has demonstrated the use of Gazebo together with ROS and PX4 to create simulation environments for UAV navigation, localization, and path-planning research [20].

Autopilot Firmware and Software-in-the-Loop (PX4 SITL)

PX4 Autopilot is an open-source flight-control software platform designed for autonomous systems including unmanned aerial vehicles (UAVs). It provides the flight-control functions required to operate a vehicle, including state estimation, control, navigation, and actuator management. PX4 can operate on physical flight-controller hardware as well as in simulation environments, allowing the same flight-control software to be evaluated before deployment on a real UAV [21].

Software-in-the-Loop (SITL) is a simulation approach in which the PX4 flight stack executes on a computer instead of a physical flight controller. The simulated vehicle provides sensor information to PX4, while PX4 processes this information and generates control and actuator outputs for the simulated vehicle. This allows flight-control software, navigation algorithms, and mission behavior to be tested without requiring physical UAV hardware [21], [22].

Robot Operating System (ROS 2)

Robot Operating System 2 (ROS 2) is a middleware framework designed to facilitate the development of distributed robotic applications. It provides a structured communication architecture through which independent software components can exchange data and coordinate robotic functions. ROS 2 was designed to address limitations of earlier robotic middleware and to support applications requiring distributed computation, real-time capabilities, and operation across different robotic platforms [23].

MAVLink Protocol

Micro Air Vehicle Link (MAVLink) is a lightweight communication protocol designed for communication between unmanned systems and external components such as ground control stations. It defines a set of messages for exchanging information related to vehicle operation, telemetry, and mission control. MAVLink has been adopted by major open-source autopilot platforms, including PX4 and ArduPilot, making it widely used in unmanned aerial vehicle (UAV) systems [24].

2.1.3.5 Centralized Observability and Dashboard Visualization with Grafana

Grafana is an open-source visualization and monitoring platform that can be used to present time-series and other monitoring data through configurable dashboards. It can be integrated with data sources such as Prometheus, allowing collected metrics to be queried and presented through graphs, tables, and other visualizations [25].

2.1.4 Existing Solutions

Recent advancements in Unmanned Aerial Vehicles (UAVs) have led to the development of autonomous systems designed to replace manual search and rescue operations. For instance, in maritime environments, autonomous UAV-based systems have been proposed to assist in time-critical Man Overboard (MOB) incidents [26]. These systems utilize onboard cameras and flight path computations to algorithmically detect humans at sea, significantly reducing the time-to-target compared to traditional coastguard vessels. However, these solutions are primarily optimized for open-water maritime environments using single-agent deployments, which lack the multi-UAV coordination required to rapidly cover massive, fragmented post-flood disaster zones.

Furthermore, contemporary research into autonomous drone platforms continues to optimize AI-based detection and cooperative search algorithms [27]. While these existing solutions successfully demonstrate the viability of airborne AI detection, they often overlook the specific operational constraints of flood environments – namely, managing dynamic battery depletion across a multi-drone fleet and the complex geometric georeferencing required when navigating over uneven, debris-filled submerged terrains.

2.2 Preliminary design based on backgrounds

[Insert Figure 2.2: Data Flow Design for the Multi-UAV Search and Rescue System here]

Figure 2.2: Data Flow Design for the Multi-UAV Search and Rescue System

Based on the study of Multi-UAV systems, PX4, ROS 2, MQTT, Kafka, computer vision, and spatiotemporal databases, a preliminary design for a Multi-UAV Search and Rescue System, as illustrated in Figure 2.2, is proposed. The system is divided into six main layers: Field and Edge, Gateway, Kafka Streaming, Processing and Decision, Data Persistence, and Presentation.

Each UAV sends telemetry data and image frames through MQTT to the Gateway Layer, where the data is forwarded to Kafka for further processing. Image frames are analyzed by YOLO to detect survivors, while the detection results are combined with UAV telemetry in the Geo-Localization Engine to estimate survivor locations.

The Multi-UAV Control component uses Mission Configuration and Drone State information to perform mCPP-based search planning, monitor UAV safety conditions, and dynamically adapt the mission when operating conditions change.

Telemetry, survivor detections, search grids, and flight paths are stored in TimescaleDB and PostGIS, while survivor snapshots are stored separately in local storage. The processed information is then visualized through Grafana using both real-time and mission summary dashboards.

Overall, the system follows a closed-loop data flow, in which UAV data is collected, processed, and analyzed to generate updated mission commands that can be sent back to the UAV fleet during the mission.

References

[1] กรมอุตุนิยมวิทยา, "ความรู้ทั่วไปเกี่ยวกับอุทกภัย," กองพยากรณ์อากาศ กรมอุตุนิยมวิทยา, 2566. [Online]. Available: https://tmd.go.th/info/อุทกภัย

[2] S. A. Sarker et al., “The First 72 Hours: Emergent Norms, Collective Efficacy and Coordination Dynamics,” IntechOpen, 2024, doi: 10.5772/intechopen.1005886. [Online]. Available: https://www.intechopen.com/online-first/1243522

[3] W. Du, G. J. FitzGerald, M. Clark, and X.-Y. Hou, “Health impacts of floods,” Prehospital and Disaster Medicine, vol. 25, no. 3, 2010, doi: 10.1017/S1049023X00008141. [Online]. Available: https://doi.org/10.1017/S1049023X00008141

[4] THE STANDARD, "ทีมกู้ภัยช่วยน้ำท่วมภาคใต้ต้องเจออุปสรรคอะไรบ้าง ทำไมการเข้าไปช่วยเหลือผู้ประสบภัยถึงยากและ
THE STANDARD News, 2566. [Online]. Available: https://www.facebook.com/thestandardth/posts/1199501572309184

[5] Mostori, "การนำโดรนมาใช้ในภารกิจกู้ภัยทางทะเลและภัยพิบัติทางน้ำ," Mostori Technology Blog, 2567. [Online]. Available: https://www.mostori.com/blog369.html

[6] E. Lygouras, N. Santavas, A. Taitzoglou, K. Tarchanidis, A. Mitropoulos, and A. Gasteratos, “Unsupervised human detection with an embedded vision system on a fully autonomous UAV for search and rescue operations,” Sensors, vol. 19, no. 16, 2019, doi: 10.3390/s19163542. [Online]. Available: https://doi.org/10.3390/s19163542

[7] G. Paulin, S. Sambolek, and M. Ivasic-Kos, “Application of raycast method for person geolocalization and distance determination using UAV images in real-world land search and rescue scenarios,” Expert Systems with Applications, vol. 237, Art. no. 121495, 2024, doi: 10.1016/j.eswa.2023.121495.

[8] Y. Yang, Z. Wei, and L. Zhu, “SL4U: A Scenario Description Language for Unmanned Swarm,” IEEE Internet of Things Journal, 2023, doi: 10.1109/JIOT.2023.3255146.

[9] S. Sambolek and M. Ivasic-Kos, “Person Detection and Geolocation Estimation in Drone Images,” SN Computer Science, vol. 6, no. 4, 2025, Art. no. 358.

[10] P. Dobbelaere and K. S. Esmaili, “Modern messaging for distributed systems,” in Proceedings of the 5th International Conference on Cloud Computing and Services Science (CLOSER), Porto, Portugal, 2017, doi: 10.5220/0006371102480258. [Online]. Available: https://www.researchgate.net/publication/277935665_Modern_Messaging_for_Distributed_Sytems

[11] OASIS Standard, “MQTT Version 5.0,” OASIS Standard Specification, 2019. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[12] MQTT.org, “MQTT: The Standard for IoT Messaging,” MQTT.org, 2026. [Online]. Available: https://mqtt.org/

[13] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A distributed messaging system for log processing,” in Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB), Athens, Greece, 2011. [Online]. Available: https://notes.stephenholiday.com/Kafka.pdf

[14] Apache Software Foundation, “Kafka Streams Architecture,” Apache Kafka Documentation, 2026. [Online]. Available: https://kafka.apache.org/documentation/streams/architecture

[15] M. A. A. da Silva, R. A. C. da Silva, and P. R. M. de Inácio, “Benchmarking Time Series Databases with IoT Data,” Sensors, vol. 20, no. 19, 2020. [Online]. Available: https://arxiv.org/pdf/1901.08304

[16] Timescale Inc., “Hypertables and Chunks,” Timescale Documentation, 2026. [Online]. Available: https://docs.timescale.com/use-timescale/latest/hypertables/

[17] Open Geospatial Consortium, “OpenGIS Implementation Standard for Geographic Information - Simple Feature Access - Part 1: Common Architecture,” OGC Standard 06-103r4, 2011. [Online]. Available: https://www.ogc.org/publications/standard/sfa/

[18] J. M. Hellerstein, J. F. Naughton, and A. Pfeffer, “Generalized search trees for database systems,” in Proceedings of the 21st International Conference on Very Large Data Bases (VLDB), Zurich, Switzerland, 1995, pp. 562–573. [Online]. Available: https://www.vldb.org/conf/1995/P562.PDF

[19] PostGIS Project, “PostGIS Reference Manual,” OSGeo, 2026. [Online]. Available: https://postgis.net/documentation/

[20] S. Chen, W. Zhou, A.-S. Yang, H. Chen, B. Li, and C.-Y. Wen, “An end-to-end UAV simulation platform for visual SLAM and navigation,” Aerospace, vol. 9, no. 2, Art. no. 48, 2022, doi: 10.3390/aerospace9020048.

[21] A. Tullu, S. Jung, S. Lee, and S. Ko, “Effects of model-specific parameters on the development of custom module in PX4 Autopilot software-in-the-loop,” International Journal of Aerospace Engineering, vol. 2025, Art. no. 4886534, 2025, doi: 10.1155/ijae/4886534.

[22] M. Abdelkader, M. Mabrok, and A. Koubaa, “OCTUNE: Optimal control tuning using real-time data with algorithm and experimental results,” Sensors, vol. 22, no. 23, Art. no. 9240, 2022, doi: 10.3390/s22239240.

[23] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” Science Robotics, vol. 7, no. 66, Art. no. eabm6074, 2022, doi: 10.1126/scirobotics.abm6074.

[24] A. Koubaa, A. Allouch, M. Alajlan, Y. Javed, A. Belghith, and M. Khalgui, “Micro Air Vehicle Link (MAVLink) in a Nutshell: A Survey,” IEEE Access, vol. 7, pp. 87658–87680, 2019, doi: 10.1109/ACCESS.2019.2924410.

[25] J. Sanches and P. R. Pereira, “Network and systems monitoring with Prometheus and Grafana,” in Proceedings of the 20th Iberian Conference on Information Systems and Technologies (CISTI 2025), vol. 1, Lecture Notes in Networks and Systems, vol. 1716, pp. 367–378, Springer, 2026, doi: 10.1007/978-3-032-10929-3_32.

[26] V. A. Feraru, R. E. Andersen, and E. Boukas, “Towards an autonomous UAV-based system to assist search and rescue operations in man overboard incidents,” in Proc. 2020 IEEE Int. Symp. Safety, Security, and Rescue Robotics (SSRR), Abu Dhabi, UAE, 2020, pp. 57–64, doi: 10.1109/SSRR50563.2020.9292583.

[27] S. W. Cho et al., “Coverage path planning for multiple UAVs in prioritized regions using ant colony optimization,” Soft Computing, vol. 30, pp. 4273–4294, 2026, doi: 10.1007/s00500-025-11020-0.